

Object-Oriented Programming in C++

Solution Handbook

IOE BCT — Year II / Part II

Prepared for Ujjwal

Chapter 2: Basics of C++ Programming

1. Default Arguments — printMessage(message, count = 1)

Default arguments let a function be called with fewer arguments than it defines; missing ones take the default value.

```
#include <iostream>
using namespace std;

void printMessage(string message, int count = 1) {
    for (int i = 0; i < count; i++)
        cout << message << endl;
}

int main() {
    printMessage("Hello IOE!");           // count defaults to 1
    printMessage("Namaste!", 3);          // overrides default, prints 3 times
    return 0;
}
```

2. Namespace Feature

Namespaces group identifiers under a name to avoid clashes between identically-named entities.

```
#include <iostream>
using namespace std;

namespace First {
    int value = 10;
    void show() { cout << "First::value = " << value << endl; }
}

namespace Second {
    int value = 20;
    void show() { cout << "Second::value = " << value << endl; }
}

int main() {
    First::show();
    Second::show();
    cout << "Sum = " << First::value + Second::value << endl;
    return 0;
}
```

3. Compound Amount $A = P(1 + r/100)^n$, default $r = 50$

```
#include <iostream>
#include <cmath>
using namespace std;

double compoundAmount(double p, int n, double r = 50) {
    return p * pow(1 + r / 100.0, n);
}

int main() {
    cout << "A = " << compoundAmount(1000, 2) << endl;           // r defaults to 50
    cout << "A = " << compoundAmount(1000, 2, 10) << endl;       // r = 10
    return 0;
}
```

4. Function Overloading with Default Arguments

```
#include <iostream>
using namespace std;

int add(int a, int b, int c = 0) {      // overload 1, with default arg
    return a + b + c;
}

double add(double a, double b) {       // overload 2
    return a + b;
}

int main() {
    cout << add(2, 3) << endl;         // int version, c = 0  -> 5
    cout << add(2, 3, 4) << endl;      // int version        -> 9
    cout << add(2.5, 3.5) << endl;    // double version     -> 6
    return 0;
}
```

Chapter 3: Objects and Classes

5. Person Class — Constructor Logic for Citizenship Number

Citizenship number is only assigned inside the constructor if age > 16.

```
#include <iostream>
#include <string>
using namespace std;

class Person {
    string name, address;
    int age;
    long citizenshipNumber;
public:
    Person(string n, int a, string addr, long citNo) {
        name = n; age = a; address = addr;
        if (age > 16)
            citizenshipNumber = citNo;
        else
            citizenshipNumber = 0;    // not eligible yet
    }
    void show() {
        cout << name << ", " << age << ", " << address << ", Citizenship: ";
        if (citizenshipNumber != 0) cout << citizenshipNumber << endl;
        else cout << "Not issued (age <= 16)" << endl;
    }
};

int main() {
    Person p1("Ujjwal", 20, "Pokhara", 123456);
    Person p2("Sujaal", 12, "Pokhara", 654321);
    p1.show();
    p2.show();
    return 0;
}
```

6. Information Class — Swap Two Objects

```
#include <iostream>
#include <string>
using namespace std;

class Information {
    string name, address;
public:
    void setData(string n, string a) { name = n; address = a; }
    void show() { cout << name << " - " << address << endl; }
    friend void swapData(Information &a, Information &b);
};

void swapData(Information &a, Information &b) {
    Information temp = a;
    a = b;
    b = temp;
}

int main() {
    Information s1, s2;
    s1.setData("Ram", "Pokhara");
    s2.setData("Shyam", "Kathmandu");
    cout << "Before swap:\n"; s1.show(); s2.show();
    swapData(s1, s2);
    cout << "After swap:\n"; s1.show(); s2.show();
    return 0;
}
```

7a. Friend Class and Friend Function

```
#include <iostream>
using namespace std;

class Box;           // forward declaration

class Inspector {    // friend class
public:
    void checkVolume(Box &b);
};

class Box {
    double length;
    friend class Inspector;           // whole class is a friend
    friend void showLength(Box &b);   // single friend function
public:
    Box(double l) : length(l) {}
};

void Inspector::checkVolume(Box &b) {
    cout << "Length via friend class = " << b.length << endl;
}

void showLength(Box &b) {
    cout << "Length via friend function = " << b.length << endl;
}

int main() {
    Box b(5.5);
    Inspector insp;
    insp.checkVolume(b);
    showLength(b);
    return 0;
}
```

7b. Time Class — Friend Function to Add Two Times (with Rollover)

```
#include <iostream>
using namespace std;

class Time {
    int hour, min, sec;
public:
    Time(int h = 0, int m = 0, int s = 0) : hour(h), min(m), sec(s) {}
    friend Time addTime(Time t1, Time t2);
    void show() { cout << hour << ":" << min << ":" << sec << endl; }
};

Time addTime(Time t1, Time t2) {
    int sec = t1.sec + t2.sec;
    int carrySec = sec / 60;
    sec %= 60;

    int min = t1.min + t2.min + carrySec;
    int carryMin = min / 60;
    min %= 60;

    int hour = t1.hour + t2.hour + carryMin;
    return Time(hour, min, sec);
}

int main() {
```

```

    Time t1(2, 45, 40), t2(1, 30, 35);
    Time t3 = addTime(t1, t2);
    t3.show();    // 4:16:15
    return 0;
}

```

7c. Friend Function Common to Two Different Classes

```

#include <iostream>
using namespace std;

class ClassB;    // forward declaration

class ClassA {
    int a;
public:
    ClassA(int x) : a(x) {}
    friend void showSum(ClassA, ClassB);
};

class ClassB {
    int b;
public:
    ClassB(int x) : b(x) {}
    friend void showSum(ClassA, ClassB);
};

void showSum(ClassA objA, ClassB objB) {
    cout << "Sum = " << objA.a + objB.b << endl;
}

int main() {
    ClassA a(10);
    ClassB b(20);
    showSum(a, b);    // 30
    return 0;
}

```

8. Animal Class — Access Specifiers Demo

```

#include <iostream>
#include <string>
using namespace std;

class Animal {
private:
    int id;
    double cost;
protected:
    string name;
public:
    void setdata(string n, int i, double c) {
        name = n; id = i; cost = c;
    }
    void showdata() {
        cout << "Name: " << name << ", ID: " << id << ", Cost: " << cost << endl;
    }
};

int main() {
    Animal a;
    a.setdata("Cow", 101, 45000);
    a.showdata();
    // a.id = 5;    // ERROR: 'id' is private, inaccessible outside the class
    return 0;
}

```

```
}
```

9a. Static Data Members and Static Member Functions

```
#include <iostream>
using namespace std;

class Counter {
    static int count;          // static data member (declared here)
public:
    Counter() { count++; }
    static int getCount() { return count; } // static member function
};

int Counter::count = 0;       // definition/initialization outside the class

int main() {
    Counter c1, c2, c3;
    cout << "Objects created: " << Counter::getCount() << endl; // 3
    return 0;
}
```

9b. Use of the 'this' Pointer

```
#include <iostream>
using namespace std;

class Box {
    int length;
public:
    Box(int length) {
        this->length = length; // 'this' resolves the naming clash
    }
    Box& setLength(int length) {
        this->length = length;
        return *this;         // returns current object for chaining
    }
    void show() { cout << "Length: " << length << endl; }
};

int main() {
    Box b(10);
    b.show();
    b.setLength(25).show();    // chained call using *this
    return 0;
}
```

Chapter 4: Operator Overloading

10. Time Class — Overload Post-Increment (++)

```
#include <iostream>
using namespace std;

class Time {
    int h, m, s;
public:
    Time(int h = 0, int m = 0, int s = 0) : h(h), m(m), s(s) {}
    Time operator++(int) {           // 'int' parameter marks this as postfix
        Time old = *this;
        s++;
        if (s == 60) { s = 0; m++; }
        if (m == 60) { m = 0; h++; }
        return old;                  // post-increment returns the OLD value
    }
    void show() { cout << h << ":" << m << ":" << s << endl; }
};

int main() {
    Time t(1, 59, 59);
    Time t2 = t++;
    t2.show();    // 1:59:59 (old value)
    t.show();     // 2:0:0 (incremented)
    return 0;
}
```

11. Matrix Class — Overload + and - (3x3)

```
#include <iostream>
using namespace std;

class Matrix {
    int m[3][3];
public:
    void input() {
        for (int i = 0; i < 3; i++)
            for (int j = 0; j < 3; j++)
                cin >> m[i][j];
    }
    Matrix operator+(Matrix b) {
        Matrix r;
        for (int i = 0; i < 3; i++)
            for (int j = 0; j < 3; j++)
                r.m[i][j] = m[i][j] + b.m[i][j];
        return r;
    }
    Matrix operator-(Matrix b) {
        Matrix r;
        for (int i = 0; i < 3; i++)
            for (int j = 0; j < 3; j++)
                r.m[i][j] = m[i][j] - b.m[i][j];
        return r;
    }
    void show() {
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) cout << m[i][j] << " ";
            cout << endl;
        }
    }
};

int main() {
    Matrix a, b;
```



```

    cout << "Enter matrix A (9 numbers): "; a.input();
    cout << "Enter matrix B (9 numbers): "; b.input();
    cout << "A + B:\n"; (a + b).show();
    cout << "A - B:\n"; (a - b).show();
    return 0;
}

```

12. Complex Numbers — +, -, *, / via Friend (Non-Member) Functions

```

#include <iostream>
using namespace std;

class Complex {
    double re, im;
public:
    Complex(double r = 0, double i = 0) : re(r), im(i) {}
    friend Complex operator+(Complex a, Complex b);
    friend Complex operator-(Complex a, Complex b);
    friend Complex operator*(Complex a, Complex b);
    friend Complex operator/(Complex a, Complex b);
    void show() { cout << re << (im >= 0 ? " + " : " - ") << abs(im) << "i" << endl; }
};

Complex operator+(Complex a, Complex b) { return Complex(a.re + b.re, a.im + b.im); }
Complex operator-(Complex a, Complex b) { return Complex(a.re - b.re, a.im - b.im); }
Complex operator*(Complex a, Complex b) {
    return Complex(a.re * b.re - a.im * b.im, a.re * b.im + a.im * b.re);
}
Complex operator/(Complex a, Complex b) {
    double denom = b.re * b.re + b.im * b.im;
    return Complex((a.re * b.re + a.im * b.im) / denom,
        (a.im * b.re - a.re * b.im) / denom);
}

int main() {
    Complex c1(3, 2), c2(1, 4);
    (c1 + c2).show();
    (c1 - c2).show();
    (c1 * c2).show();
    (c1 / c2).show();
    return 0;
}

```

13a. Distance (Meter/Centimeter) — Overload >

```

#include <iostream>
using namespace std;

class Distance {
    int meter, cm;
public:
    Distance(int m = 0, int c = 0) : meter(m), cm(c) {}
    int toCm() { return meter * 100 + cm; }
    bool operator>(Distance d) {
        return toCm() > d.toCm();
    }
};

int main() {
    Distance d1(5, 40), d2(4, 90);
    if (d1 > d2) cout << "d1 is greater than d2" << endl;
    else cout << "d2 is greater or equal" << endl;
    return 0;
}

```

13b. Amount (Rupee/Paisa) — Overload >

```
#include <iostream>
using namespace std;

class Amount {
    int rupees, paisa;
public:
    Amount(int r = 0, int p = 0) : rupees(r), paisa(p) {}
    double toPaisa() { return rupees * 100 + paisa; }
    bool operator>(Amount b) {
        return toPaisa() > b.toPaisa();
    }
};

int main() {
    Amount a1(500, 50), a2(499, 99);
    cout << (a1 > a2 ? "a1 is greater" : "a2 is greater or equal") << endl;
    return 0;
}
```

14. Date Class — Overload - (Difference in Days)

Uses a simplified day count (365 days/year, 30 days/month) to keep the operator-overloading concept clear.

```
#include <iostream>
using namespace std;

class Date {
    int year, month, day;
public:
    Date(int y, int m, int d) : year(y), month(m), day(d) {}
    long toDays() const {
        return year * 365L + month * 30L + day;
    }
    long operator-(Date d) {
        return toDays() - d.toDays();
    }
};

int main() {
    Date d1(2026, 7, 21), d2(2025, 1, 1);
    cout << "Difference: " << (d1 - d2) << " days" << endl;
    return 0;
}
```

15. Cartesian to Polar Conversion (Logic in Destination Class)

```
#include <iostream>
#include <cmath>
using namespace std;

class Polar;    // forward declaration

class Cartesian {
    double x, y;
public:
    Cartesian(double x, double y) : x(x), y(y) {}
    double getX() { return x; }
    double getY() { return y; }
};

class Polar {
    double r, theta;
public:
```

```
Polar(Cartesian c) { // conversion constructor in destination class
    r = sqrt(c.getX() * c.getX() + c.getY() * c.getY());
    theta = atan2(c.getY(), c.getX()) * 180 / M_PI;
}
void show() { cout << "r = " << r << ", theta = " << theta << " degrees" << endl; }
};

int main() {
    Cartesian c(3, 4);
    Polar p = c; // implicit conversion via Polar(Cartesian)
    p.show(); // r = 5, theta = 53.13
    return 0;
}
```

Chapter 5: Inheritance

16. Cricketer, Bowler, Batsman — Function Overriding

```
#include <iostream>
#include <string>
using namespace std;

class Cricketer {
protected:
    string name;
public:
    Cricketer(string n) : name(n) {}
    virtual void showStats() {
        cout << name << ": generic cricketer stats" << endl;
    }
};

class Bowler : public Cricketer {
    int wickets;
public:
    Bowler(string n, int w) : Cricketer(n), wickets(w) {}
    void showStats() override {
        cout << name << " (Bowler): Wickets = " << wickets << endl;
    }
};

class Batsman : public Cricketer {
    int runs;
public:
    Batsman(string n, int r) : Cricketer(n), runs(r) {}
    void showStats() override {
        cout << name << " (Batsman): Runs = " << runs << endl;
    }
};

int main() {
    Bowler b("Sandeep", 45);
    Batsman s("Rohit", 5000);
    b.showStats();
    s.showStats();
    return 0;
}
```

17. Order of Constructor/Destructor Invocation (Multiple Inheritance)

```
#include <iostream>
using namespace std;

class A {
public:
    A() { cout << "A constructor\n"; }
    ~A() { cout << "A destructor\n"; }
};

class B {
public:
    B() { cout << "B constructor\n"; }
    ~B() { cout << "B destructor\n"; }
};

class C : public A, public B {    // multiple inheritance
public:
    C() { cout << "C constructor\n"; }
    ~C() { cout << "C destructor\n"; }
```

```
};

int main() {
    C obj;
    // Construction order: A -> B -> C
    // Destruction order (reverse): C -> B -> A
    return 0;
}
```

18. Student -> Test -> Result (Multilevel Inheritance)

```
#include <iostream>
#include <string>
using namespace std;

class Student {
protected:
    string name;
    int roll;
public:
    Student(string n, int r) : name(n), roll(r) {}
};

class Test : public Student {
protected:
    int marks1, marks2;
public:
    Test(string n, int r, int m1, int m2) : Student(n, r), marks1(m1), marks2(m2) {}
};

class Result : public Test {
public:
    Result(string n, int r, int m1, int m2) : Test(n, r, m1, m2) {}
    void display() {
        int total = marks1 + marks2;
        cout << "Name: " << name << ", Roll: " << roll
            << ", Total: " << total << endl;
    }
};

int main() {
    Result r("Aashish", 5, 78, 85);
    r.display();
    return 0;
}
```

19. Multipath Inheritance — Ambiguity Resolution (Virtual Base Class)

```
#include <iostream>
#include <string>
using namespace std;

class Person {
public:
    string name = "Unknown";
};

class Student : virtual public Person {}; // virtual inheritance
class Employee : virtual public Person {}; // avoids duplicate Person copies

class TeachingAssistant : public Student, public Employee {};

int main() {
    TeachingAssistant ta;
```

```
    ta.name = "Priya";           // no ambiguity, thanks to the virtual base class
    cout << ta.name << endl;
    return 0;
}
```

20. Manager Class — Use of Virtual Destructor

```
#include <iostream>
using namespace std;

class Employee {
public:
    virtual ~Employee() { cout << "Employee destructor\n"; }    // virtual!
};

class Student {
public:
    virtual ~Student() { cout << "Student destructor\n"; }
};

class Manager : public Employee, public Student {
public:
    ~Manager() { cout << "Manager destructor\n"; }
};

int main() {
    Employee *e = new Manager();
    delete e;
    // Because ~Employee() is virtual, this correctly invokes
    // Manager -> Student -> Employee destructors, not just Employee's
    return 0;
}
```

Extended Topics: File I/O, Templates, and Exceptions

21. File I/O — Copy File, Converting Lowercase to Uppercase

```
#include <iostream>
#include <fstream>
#include <cctype>
using namespace std;

int main() {
    ifstream fin("source.txt");
    ofstream fout("dest.txt");
    char ch;

    if (!fin) { cout << "Cannot open source file\n"; return 1; }

    while (fin.get(ch))
        fout.put(toupper(ch));

    fin.close();
    fout.close();
    cout << "File copied with uppercase conversion." << endl;
    return 0;
}
```

22. File I/O — Store and Search Employee Records in a Binary File

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

class Employee {
    int id;
    char name[30];
public:
    void getData() {
        cout << "Enter ID: "; cin >> id;
        cout << "Enter Name: "; cin >> name;
    }
    void showData() { cout << id << " - " << name << endl; }
    int getId() { return id; }
};

void writeRecord() {
    ofstream fout("employee.dat", ios::binary | ios::app);
    Employee e;
    e.getData();
    fout.write((char*)&e, sizeof(e));
    fout.close();
}

void searchRecord(int searchId) {
    ifstream fin("employee.dat", ios::binary);
    Employee e;
    bool found = false;
    while (fin.read((char*)&e, sizeof(e))) {
        if (e.getId() == searchId) { e.showData(); found = true; break; }
    }
    if (!found) cout << "Record not found" << endl;
    fin.close();
}

int main() {
    int choice, id;
```

```

do {
    cout << "1. Add Employee  2. Search  0. Exit\n";
    cin >> choice;
    if (choice == 1) writeRecord();
    else if (choice == 2) { cout << "Enter ID to search: "; cin >> id; searchRecord(id); }
} while (choice != 0);
return 0;
}

```

23. Complex Numbers — Overload << and >>

```

#include <iostream>
using namespace std;

class Complex {
    double re, im;
public:
    Complex(double r = 0, double i = 0) : re(r), im(i) {}
    friend istream& operator>>(istream &in, Complex &c) {
        cout << "Enter real and imaginary parts: ";
        in >> c.re >> c.im;
        return in;
    }
    friend ostream& operator<<(ostream &out, Complex &c) {
        out << c.re << " + " << c.im << "i";
        return out;
    }
};

int main() {
    Complex c;
    cin >> c;
    cout << "You entered: " << c << endl;
    return 0;
}

```

24. Template — Read n Numbers of Any Type, Sort Descending

```

#include <iostream>
using namespace std;

template <class T>
void sortDescending(T arr[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (arr[j] < arr[j + 1])
                swap(arr[j], arr[j + 1]);
}

template <class T>
void display(T arr[], int n) {
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;
}

int main() {
    int a[] = {4, 1, 9, 3};
    double b[] = {2.2, 8.8, 1.1};

    sortDescending(a, 4);
    display(a, 4);          // 9 4 3 1

    sortDescending(b, 3);
    display(b, 3);          // 8.8 2.2 1.1
    return 0;
}

```



```
}
```

25. Function Template — Not All Template Parameters Used in Arguments

```
#include <iostream>
using namespace std;

template <class T, class U>    // U does not appear in the function's parameters
T convert(U value) {
    return static_cast<T>(value);
}

int main() {
    double d = 9.75;
    int i = convert<int>(d);    // T explicitly given as int, U deduced as double
    cout << i << endl;        // 9
    return 0;
}
```

26. Default Arguments Within a Class Template

```
#include <iostream>
using namespace std;

template <class T = int>
class Box {
    T value;
public:
    Box(T v = T()) : value(v) {}    // default argument inside a template class
    void show() { cout << "Value: " << value << endl; }
};

int main() {
    Box<> b1;    // T defaults to int, value defaults to 0
    Box<double> b2(3.14);
    b1.show();
    b2.show();
    return 0;
}
```

27. Exception Handling — Multiple Exceptions (Negative Roll No. / Marks Exceeding Limit)

```
#include <iostream>
using namespace std;

class NegativeRollException {};
class MarksExceedException {};

void validate(int roll, int marks) {
    if (roll < 0) throw NegativeRollException();
    if (marks > 100) throw MarksExceedException();
    cout << "Valid record: Roll " << roll << ", Marks " << marks << endl;
}

int main() {
    try {
        validate(-5, 80);
    } catch (NegativeRollException&) {
        cout << "Error: Roll number cannot be negative" << endl;
    } catch (MarksExceedException&) {
        cout << "Error: Marks cannot exceed 100" << endl;
    }
}
```

```

    try {
        validate(10, 150);
    } catch (NegativeRollException&) {
        cout << "Error: Roll number cannot be negative" << endl;
    } catch (MarksExceedException&) {
        cout << "Error: Marks cannot exceed 100" << endl;
    }
    return 0;
}

```

28. Triangle Area — Exception for Invalid Sides, with Re-throw

```

#include <iostream>
#include <cmath>
#include <stdexcept>
using namespace std;

double triangleArea(double a, double b, double c) {
    if (a + b <= c || b + c <= a || a + c <= b)
        throw invalid_argument("Invalid triangle sides");
    double s = (a + b + c) / 2;
    return sqrt(s * (s - a) * (s - b) * (s - c));
}

void wrapper(double a, double b, double c) {
    try {
        cout << "Area = " << triangleArea(a, b, c) << endl;
    } catch (invalid_argument &e) {
        cout << "Caught in wrapper, rethrowing..." << endl;
        throw; // re-throw to caller
    }
}

int main() {
    try {
        wrapper(1, 2, 10); // invalid sides
    } catch (invalid_argument &e) {
        cout << "Caught in main: " << e.what() << endl;
    }
    return 0;
}

```

29. Square Root — Exception for Negative Input

```

#include <iostream>
#include <cmath>
#include <stdexcept>
using namespace std;

double safeSqrt(double x) {
    if (x < 0) throw runtime_error("Cannot compute square root of a negative number");
    return sqrt(x);
}

int main() {
    double num;
    cout << "Enter a number: ";
    cin >> num;
    try {
        cout << "Square root = " << safeSqrt(num) << endl;
    } catch (runtime_error &e) {
        cout << "Error: " << e.what() << endl;
    }
    return 0;
}

```


Polymorphism

30. Abstract Class Shape — Circle, Rectangle, Square

```
#include <iostream>
using namespace std;

class Shape {
public:
    virtual double area() = 0;    // pure virtual function -> makes class abstract
    virtual void draw() { cout << "Drawing a shape" << endl; }
};

class Circle : public Shape {
    double radius;
public:
    Circle(double r) : radius(r) {}
    double area() override { return 3.1416 * radius * radius; }
    void draw() override { cout << "Drawing a circle" << endl; }
};

class Rectangle : public Shape {
    double length, width;
public:
    Rectangle(double l, double w) : length(l), width(w) {}
    double area() override { return length * width; }
    void draw() override { cout << "Drawing a rectangle" << endl; }
};

class Square : public Shape {
    double side;
public:
    Square(double s) : side(s) {}
    double area() override { return side * side; }
    void draw() override { cout << "Drawing a square" << endl; }
};

int main() {
    Shape *shapes[3];
    shapes[0] = new Circle(5);
    shapes[1] = new Rectangle(4, 6);
    shapes[2] = new Square(3);

    for (int i = 0; i < 3; i++) {
        shapes[i]->draw();
        cout << "Area = " << shapes[i]->area() << endl;
        delete shapes[i];
    }
    return 0;
}
```

31. Run-time Polymorphism — Late Binding with Virtual Functions

```
#include <iostream>
using namespace std;

class Base {
public:
    virtual void show() { cout << "Base class show()" << endl; }    // enables late binding
};

class Derived : public Base {
public:
    void show() override { cout << "Derived class show()" << endl; }
};
```

```
int main() {  
    Base *bp;  
    Base b;  
    Derived d;  
  
    bp = &b;  
    bp->show();    // Base class show()  
  
    bp = &d;  
    bp->show();    // Derived class show() -- resolved at RUN TIME (late binding)  
    return 0;  
}
```